Лабораторная работа: управление уязвимостями в ОС Linux

Тема: обнаружение, приоритизация и устранение уязвимостей на примере цепочки CVE в системе печати CUPS.

Среда выполнения: виртуальная машина VirtualBox / VMware / QEMU, Debian 13 (Trixie). Допускается любой Linux-дистрибутив с Docker (Debian, Ubuntu, Astra и т. п.) — учебная ВМ не обязательна.

Формат: «инвентаризация → обнаружение → приоритизация → устранение → проверка», как в реальном процессе управления уязвимостями (vulnerability management).

Что нужно для зачёта

№	Задание	Статус
1	Инвентаризация и обнаружение уязвимого состояния	**обязательно**
2	Приоритизация: что и в каком порядке исправлять	**обязательно**
3	Устранение, проверка и сохранение исправления после перезапуска	**обязательно**
4	Аудит и журналирование (nftables log / Wazuh)	**по желанию**

По каждому заданию нужны скриншоты с видимой меткой `[LAB:...]` (см. п. 0.4). Задание 4 не влияет на зачёт: это материал для темы «аудит и расследование инцидентов» и задел под работу с Wazuh.

Виртуальная машина

- **Готовый образ ВМ (рекомендуется):** скачать
- **ОС:** Debian 13 (Trixie), минимальная установка, без десктопа
- **Логин:** `root`
- **Пароль:** `qwe123!@#`
- **Сеть:** достаточно NAT — стенд работает внутри ВМ

0. Общие сведения

0.1. Зачем нужен отдельный стенд

В Debian 13 установлены уже **исправленные** версии CUPS: уязвимых пакетов в репозитории просто нет. Это нормальная ситуация — и одновременно главный практический вывод курса: уязвимость существует не «в системе вообще», а в **конкретной версии конкретного пакета**. Поэтому уязвимое состояние воспроизводится в контейнере, где версии зафиксированы.

Контейнер здесь — не «просто удобно», а единственный способ получить уязвимые версии пользовательского ПО, не ломая основную систему. Ядро при этом используется хостовое (Debian 13), поэтому уязвимости ядра в этой работе не рассматриваются — они требуют отдельной ВМ со своим ядром.

0.2. Подготовка стенда

Скрипт подготовки лежит в репозитории курса. Скачайте его на ВМ и запустите один раз:

```
mkdir -p ~/lab_vuln && cd ~/lab_vuln

REPO=https://raw.githubusercontent.com/james116blue/os-protection/2026_fall
wget -O lab_vuln_setup.sh \
     "$REPO/lab_vulnerability_management/scripts/lab_vuln_setup.sh"

chmod +x lab_vuln_setup.sh
sudo ./lab_vuln_setup.sh "Ivanov_I_I"      # укажите свою фамилию и инициалы
```

> Если `wget` не установлен: `apt-get update && apt-get install -y wget` (или замените `wget -O` на `curl -fsSL -o`).

Скрипт печатает метку лабораторной работы и подсказки. Повторный запуск безопасен: контейнер пересоздаётся, метка генерируется заново.

0.3. Что делает скрипт подготовки

Шаг	Действие
1	Устанавливает `docker.io`, включает службу `docker`
2	Генерирует уникальную метку `LAB_TOKEN` и встраивает её в приглашение терминала
3	Скачивает файлы стенда в `/opt/lab-cups`
4	Получает образ `cups-lab:1.0`: из готового файла `/opt/lab-cups/cups-lab-1.0.tar`, из registry или собирает на месте
5	Запускает контейнер `cups-lab` в уязвимом состоянии

Сборка образа на месте (если готового файла нет) занимает 3–5 минут и требует доступа в интернет. Скачивать и собирать что-либо вручную не нужно.

0.4. Правило анти-плагиата

На каждом скриншоте должна быть видна метка `[LAB:xxxxxxxxxxxxxxxx]` — в приглашении терминала или в выводе скриптов. Метка вычисляется из вашей фамилии, имени хоста, времени запуска и случайного числа, поэтому у каждого студента она своя.

Посмотреть метку:

```
cat /etc/lab-token
echo "$LAB_TOKEN"
docker exec cups-lab cat /etc/lab-token
```

Откройте новый терминал (или выполните `exec bash -l`), чтобы метка появилась в приглашении. Скриншоты без метки, с чужой меткой или с меткой, не совпадающей с вашим запуском скрипта, не засчитываются.

0.5. Проверка, что стенд поднялся

```
docker ps                                  # контейнер cups-lab в состоянии Up
docker exec -it cups-lab bash              # вход внутрь
check_vuln.sh                              # должен найти признаки уязвимости
```

Внутри контейнера уже доступны `check_vuln.sh` и `fix_vuln.sh` — это те же действия, которые вы будете выполнять командами вручную.

0.6. Безопасность стенда

Порты опубликованы **только на loopback хоста**: `127.0.0.1:1631/udp → 631/udp` (cups-browsed) и `127.0.0.1:1632/tcp → 631/tcp` (cupsd). Стенд не открыт в сеть и не должен в неё открываться: уязвимая служба, доступная из сети, — это уже не учебный стенд, а инцидент.

Теория: что именно мы делаем

CVE — идентификатор конкретной уязвимости. **CVSS** — оценка её технической тяжести (0–10), но не вероятности эксплуатации и не важности для вашего актива. **EPSS** — прогноз вероятности эксплуатации в ближайшие 30 дней. **KEV** (CISA) — список уязвимостей, которые уже эксплуатируются.

Отсюда практическое правило: **приоритет ≠ CVSS**. В расчёт идут:

1. **Экспозиция** — служба слушает на всех интерфейсах или только на `127.0.0.1`; есть ли до неё путь из недоверенной сети.
2. **Доступность исправления** — есть ли обновление, требует ли оно перезагрузки, ломает ли совместимость.
3. **Критичность актива** — что будет, если сработает (компрометация рабочей станции и компрометация контроллера домена — разные события).
4. **Наличие компенсирующих мер** — можно ли закрыть риск быстрее, чем выйдет патч (отключить службу, сузить привязку, закрыть порт firewall).

Жизненный цикл: **инвентаризация → обнаружение → приоритизация → устранение → проверка**. Последний шаг обязателен: «обновили пакет» — не то же самое, что «уязвимость закрыта».

Кейс CUPS. Цепочка CVE-2024-47076 / 47175 / 47176 / 47177: `cups-browsed` принимает анонсы принтеров по UDP/631 от любого источника (47176), полученные IPP-атрибуты не проверяются (47076) и попадают в PPD-файл (47175), где может оказаться директива `FoomaticRIPCommandLine` с командой (47177). Итог — удалённое выполнение кода при печати. Уязвимы `cups-browsed` ≤ 2.0.1, `libcupsfilters` и `libppd` ≤ 2.1b1, `cups-filters` ≤ 2.0.1.

Полная теория — в лекции занятия: `занятие 4 …/лекция/лекция_4.md`.

Задание 1 (обязательно). Инвентаризация и обнаружение

Цель: доказать, что стенд уязвим, и собрать инвентаризацию.

Шаг 1. Автоматическая проверка

```
docker exec -it cups-lab bash
check_vuln.sh
```

Скрипт проверяет пять признаков: версии пакетов (сравнение с исправленными через `dpkg --compare-versions`), запущенный `cups-browsed`, настройку `BrowseRemoteProtocols`, прослушивание UDP/631 и наличие правил фильтрации. Код возврата 1 означает «уязвимо», `0` — «признаков нет».

```
check_vuln.sh; echo "код возврата: $?"
```

Шаг 2. Ручная инвентаризация

Проверьте то же самое «руками» — отчёт должен опираться на ваши команды, а не только на готовый скрипт:

```
dpkg -l | grep -E 'cups|libppd'
apt-cache policy cups-browsed cups-filters libcupsfilters2t64 libppd2

pgrep -a cups-browsed
ss -ulpn | grep ':631'
grep -E '^(BrowseRemoteProtocols|BrowseLocalProtocols)' /etc/cups/cups-browsed.conf
```

Обратите внимание на `apt-cache policy`: там видно, какая версия установлена и какая доступна в репозитории. Это и есть ответ на вопрос «есть ли фикс».

Шаг 3. Таблица уязвимостей

Для каждого из четырёх компонентов определите: установленная версия, исправленная версия, CVE, оценка CVSS (источник — NVD), что даёт эксплуатация.

Компонент	Версия	Исправлено в	CVE	CVSS
cups-browsed			CVE-2024-47176	
libcupsfilters			CVE-2024-47076	
libppd			CVE-2024-47175	
cups-filters			CVE-2024-47177	

> **Скриншоты 1.1–1.3:** вывод `check_vuln.sh` (все `[УЯЗВИМО]`); вывод `dpkg -l` и `apt-cache policy`; вывод `ss -ulpn` и `pgrep -a cups-browsed`. Метка `[LAB:...]` должна быть видна на каждом.

Задание 2 (обязательно). Приоритизация

Цель: обосновать порядок действий, а не просто «обновить всё».

Шаг 1. Определите экспозицию

```
# внутри контейнера
ip -4 addr show
ss -ulpn | grep ':631'
```

`0.0.0.0:631` означает, что служба принимает пакеты со всех интерфейсов, а не только с `127.0.0.1`. Сформулируйте: доступна ли служба из недоверенной сети в вашей конфигурации и что изменится, если контейнер опубликовать наружу.

Шаг 2. Постройте таблицу приоритетов

Для каждой уязвимости укажите:

- **вектор** (сеть / локально, нужна ли аутентификация);
- **условия эксплуатации** (нужно ли, чтобы пользователь отправил задание на печать);
- **есть ли фикс** и требует ли он перезапуска службы;
- **компенсирующая мера**, которую можно применить прямо сейчас.

Шаг 3. Обоснуйте порядок

Напишите 5–7 строк: что делаете первым и почему. Ожидаемая логика: сетевой вектор (`cups-browsed`, UDP/631) закрывается быстрее всего отключением службы и firewall — это не требует простоя печати; обновление пакетов закрывает всю цепочку, но требует доступа к репозиторию и перезапуска служб. Порядок «сначала быстрая компенсирующая мера, затем патч» — нормальная практика, а не компромисс.

> **Скриншот 2.1:** вывод `ip -4 addr` и `ss -ulpn` с видимым `0.0.0.0:631`. Таблица приоритетов — текстом в отчёте.

Задание 3 (обязательно). Устранение, проверка, персистентность

Цель: привести стенд в защищённое состояние и доказать это.

Шаг 1. Устранение

`fix_vuln.sh`

Скрипт делает четыре вещи (разберите каждую):

1. обновляет `cups`, `cups-browsed`, `cups-filters`, `libcupsfilters2t64`, `libppd2` до исправленных версий;
2. выставляет `BrowseRemoteProtocols none` в `/etc/cups/cups-browsed.conf`;
3. останавливает `cups-browsed` и ставит флаг `/etc/lab-hardened`, чтобы служба не поднялась при следующем старте;
4. добавляет правила nftables: `udp dport 631 counter drop` и `tcp dport 631 ip saddr != 127.0.0.1 counter drop`.

Шаг 2. Проверка состояния

```
check_vuln.sh; echo "код возврата: $?"     # ожидается 0 и «признаков не найдено»
ss -ulpn | grep ':631' || echo 'UDP/631 не слушается'
pgrep -a cups-browsed || echo 'cups-browsed не запущен'
```

Шаг 3. Проверка firewall по счётчикам

Счётчик — объективное доказательство, что пакеты действительно попадают под правило (а не «правило вроде есть»).

```
# на хосте (в ВМ, вне контейнера)
echo test | nc -u -w1 127.0.0.1 1631
docker exec cups-lab nft list table inet lab
```

Число пакетов у правила `udp dport 631 counter drop` должно увеличиться.

Шаг 4. Проверка, что функция не сломана

Защита не должна ломать печать: cupsd продолжает работать.

```
docker exec cups-lab lpstat -r        # «scheduler is running»
```

Шаг 5. Персистентность

```
docker restart cups-lab
docker exec -it cups-lab bash
check_vuln.sh                              # снова 0
ss -ulpn | grep ':631' || echo 'UDP/631 не слушается'
```

Если после перезапуска стенд снова уязвим — исправление было неперсистентным. Именно так в реальных системах «закрытая» уязвимость возвращается после обновления пакета, который перезаписал конфигурацию.

> **Скриншоты 3.1–3.4:** вывод `fix_vuln.sh`; `check_vuln.sh` с кодом 0; счётчики nftables после отправки пакетов; `check_vuln.sh` после `docker restart`.

Задание 4 (по желанию). Аудит и журналирование

Цель: превратить защитную меру в источник телеметрии — основу расследования инцидентов.

Вариант A. Журнал firewall (nftables log)

Добавьте правило с логированием **перед** правилом блокировки (правило без verdict только пишет в журнал и передаёт пакет дальше):

```
docker exec cups-lab nft insert rule inet lab input \
    udp dport 631 counter log prefix "CUPS-DROP: "
```

Отправьте пакеты с хоста:

```
echo test | nc -u -w1 127.0.0.1 1631
```

Смотрите журнал ядра на хосте:

```
journalctl -k --since "5 min ago" | grep CUPS-DROP
```

Вы увидите, что попадание под правило фиксируется с префиксом, адресом источника и портом. Это ровно тот тип события, который нужен SIEM.

Вариант B. Wazuh: правило и алерт

Если у вас есть стенд Wazuh (готовая ВМ и инструкция — в материалах курса по аудиту), подключите эту ВМ агентом и добавьте правило в `/var/ossec/etc/rules/local_rules.xml`:

```
<group name="local,cups,">
  <rule id="100631" level="7">
    <match>CUPS-DROP</match>
    <description>CUPS: обращение к UDP/631 заблокировано firewall (возможный скан или эксплуа
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
</group>
```

Проверьте правило на одной строке через `wazuh-logtest`, затем повторите отправку пакетов и найдите алерт в Dashboard.

> **Скриншот 4.1:** строка журнала `CUPS-DROP` (вариант A) или алерт в Wazuh (вариант B).

Итоговая форма отчёта

> **Обязательны задания 1, 2 и 3.** Задание 4 — по желанию.

Для каждого задания в отчёт включаются:

1. Номер и название задания.
2. Скриншоты с меткой `[LAB:...]`.
3. Команды, которые вы выполняли (в тексте, не только на скриншотах).
4. Краткий вывод: что обнаружено / почему такой порядок / что подтвердила проверка.

№	Задание	Статус	Результат
1	Обнаружение	обязательное	таблица CVE + `check_vuln.sh` = 1
2	Приоритизация	обязательное	обоснованный порядок действий
3	Устранение и проверка	обязательное	`check_vuln.sh` = 0, счётчики, персистентность
4	Аудит и логи	по желанию	событие `CUPS-DROP` / алерт Wazuh

Отчёты без видимой на всех скриншотах метки `LAB_TOKEN` либо с меткой, не совпадающей с выданной вашему запуску `lab_vuln_setup.sh`, к проверке не принимаются.

Состав пакета файлов

```
lab_vulnerability_management/scripts/lab_vuln_setup.sh  — подготовка стенда + метка LAB_TOKEN
lab_vulnerability_management/scripts/build_cups_lab_image.sh — сборка образа (для преподавателя
lab_vulnerability_management/cups-lab/Dockerfile      — образ с зафиксированными уязвимыми вер
lab_vulnerability_management/cups-lab/check_vuln.sh     — обнаружение (5 признаков)
lab_vulnerability_management/cups-lab/fix_vuln.sh     — устранение + персистентность
lab_vulnerability_management/cups-lab/entrypoint.sh    — запуск уязвимого/защищённого состояни
lab_vulnerability_management/cups-lab/docker-compose.yml — альтернативный запуск через compose
```